Solution 1. The user should provide $S \subseteq V(D)$, such that $\delta^{\text{out}}(S) = B$ and $\delta^{\text{in}}(S) = \emptyset$, where B is the directed cut. This object must be easy to have when the user provides a directed cut.

Algorithm 1 Verify-Directed-Cut(D, B, S)

```
for each a ∈ A(D) do
    v, u ← ends[a]
    if v ∈ S and u ∉ S then    ▷ This can be done in constant time using an auxiliary array
        if a ∉ B then                                   ▷ Same thing from above comment
            return False
        end if
    else if v ∉ S and u ∈ S then
        return False
    else if v ∈ S and u ∈ S then
        if a ∈ B then
            return False
        end if
    else
        if a ∈ B then
            return False
        end if
    end if
end for
return True
```

It is essential to state that building the auxiliary arrays mentioned on the pseudocode costs linear time on the size of the associated sets. The algorithm checks the given definition of directed cut in question 9. By that, we also know that an arc between two vertices in S can not be in B, the same for an arc between two vertices that are not in S.

Solution 2. I will prove that $H_1^{\min} = H_2^{\min}$ by showing that $H_1^{\min} \subseteq H_2^{\min}$ and that $H_2^{\min} \subseteq H_1^{\min}$. I will show that $X \subseteq Y$ by proving that an arbitrary element $x_0 \in X$ lies in Y.

Given $h_1 \in H_1^{\min}$, I will prove that $h_1 \in H_2$ and then that $h_1 \in H_2^{\min}$. h_1 is the set of all the outwards arcs from a set of vertices X, that includes r and excludes s. Given that it is impossible to have an rs-path if there are no arcs from a set that includes r to the other vertices of $V(D)$ that includes s, $h_1 \subseteq H_2$.

h_1 is minimal in H_2 if every element in h_1 is an arc that can be in a different rs-path from all other arcs in h_1. If there is an arc that is only in the same rs-paths from another arc in h_1, it could be removed, and this new set would be a proper element of H_2, and h_1 would not be minimal.

h_1 does not have more than one arc sharing only same rs-paths because it would not be minimal in $H_1^{\min}$. This holds because you could remove from h_1 all the arcs in those same rs-paths besides the outward arcs of the closest vertex to r that is in a shore (of this rs-cut). After removing the arc (s), the set would continue to be an rs-cut (a possible shore is to maintain a previous shore and remove all the vertices along the path that were in the previous shore and are between the vertex closer to r and the farthest one in the path).

Adding to the discussion, it is not possible to have an arc in h_1 that leads to a path that does not reach s because this is not a minimal rs-cut. Given that you could add all the vertices of those dead-end paths to a previous shore to have a new shore associated with an rs-cut that is a subset of the previous rs-cut. Finally, there are no arcs in h_1 that are part of an extension (loops) to an $rs - path$ with another arc in h_1. In this scenario, you could remove it by having a new shore that adds all the vertices of the loops (and possible dead end walks) to a shore associated with the previous rs-cut.Given all the arguments, $h_1 \subseteq H_2$, and it is minimal in H_2, thus $H_1^{\min} \subseteq H_2^{\min}$.

Given $h_2 \in H_2^{\min}$, I will prove that $h_2 \in H_1$ and then that $h_2 \in H_1^{\min}$. h_2 is a minimal subset $B \subseteq A$ such that $D - B$ has no rs-path, which is an rs-cut. A possible shore to this rs-cut is the set of all the vertices $v \in V(D)$ and connected from r to the vertices that the outward edges are on B (I will explain that you may have to add some vertices to the set). If v has outward arcs that are not in B, it means that it is an arc that does not lead to a new path to s or is part of a loop that goes back to a vertex in the path which the arc that is in B is part of. In both cases, because of the arguments I explained before, you could have a new shore, including all the vertices of the loop and the paths that do not reach s, which is associated with a new rs-cut without those arcs that are not in B. Thus, $h_2 \subseteq H_1$. Also, h_2 is minimal in H_1 because if you remove an arc a_2 from h_2, you can not have the rs-cut $h_2 - a_2$ (there is no shore associated with it). This holds because this arc is the unique arc in h_2 from an rs-path, and h_2 is minimal in H_2. If you add any neighbor of the tail of a_2 in a h_2 shore to try to remove a_2 from h_2, you will add at least one arc in h_2 because the end of the rs-path is s and s can not be in a shore. If you try to remove vertices from a shore associated with h_2, you would add at least one arc to h_2 because the beginning of the rs-path is r, and r is in every possible shore. In both cases, you can not remove an arc from h_2 without adding another arc. Given that, $h_2 \subseteq H_1$, and it is minimal in H_1, thus $H_2^{\min} \subseteq H_1^{\min}$.

By the arguments shown, $H_1^{\min} = H_2^{\min}$.

Solution 3. The following algorithm solves the problem.

Algorithm 2 MinimizeWalk(D, r, s)

```
V_aux := ∅                                   ▷ set of vertices of an auxiliary digraph
A_aux := ∅                                   ▷ set of arcs of an auxiliary digraph
for each v ∈ V(D) do
    V_aux ← V_aux ∪ (v,0) ∪ (v,1) ∪ (v,2)    ▷ new vertices on the auxiliary digraph
end for
for each a ∈ V(A) do
    v, u ← ends[a]
    A_aux ← A_aux ∪ ((v,0),(u,1)) ∪ ((v,1),(u,2)) ∪ ((v,2),(u,0))
end for
π, d := Breadth-First-Search((V_aux, A_aux), v)
if d[s_1] < ∞ then
    p_aux := Path-From-Branching((π, d), s)
    p := Relabeling(p_aux)                   ▷ This function will be explained below
    return p
end if
return NIL
```

Using the TA's hint, I will use an auxiliary digraph $D_{\text{aux}} := (V_{\text{aux}}, A_{\text{aux}})$ to solve the problem. D_{aux} is built from D and based on two rules:

- For each $v \in V(D)$, V_{aux} has three vertices v_0, v_1, v_2, so the size of V_{aux} is three times the size of $V(D)$.
- For each arc $vu \in A(D)$, A_{aux} has three arcs v_0u_1, v_1u_2, v_2u_0. Also, A_{aux} is three times the size of $A(D)$.

The algorithm builds the auxiliary digraph, runs a Breadth-First-Search on the auxiliary digraph, and then translates the answer to the desired result.

After building this auxiliary digraph, we run a Breadth-First-Search and verify if there is a path from r_0 to s_1. The idea to do that is that every path on the auxiliary digraph that ends on a vertex of the form $(v, 1)$ is a path where the length $l \equiv 1 \pmod 3$. This is evident because, on the auxiliary digraph, for every arc that you traverse, you go from a vertex of the form $(v, i), i \in \{0, 1, 2\}$ to a vertex of the form $(u, (i+1)\%3)$, where $\%$ is the operator that gives you the remainder of the division. As for each arc that you traverse, the length of the path increases 1, if you start a walk w at a vertex of the form $(v, 0)$, for each vertex (v, i) of the walk, holds that the length of w till this vertex is equivalent to i mod 3. By that, if we end on a vertex of the form $(v, 1)$, the length l of the path on the auxiliary digraph is equivalent to 1 mod 3.

Also, if there is an rs-walk, where the length $l \equiv 1 \pmod 3$, this algorithm will find it. Firstly, every walk in D has a correspondent walk in the auxiliary digraph because if you can go from a vertex v to a vertex u on D, you can go from a vertex of the form $(v, i), i \in \{0, 1, 2\}$ to a vertex of the form $(u, (i+1)\%3)$ in the auxiliary digraph. If you ignore the second element, you will have a walk that can be easily converted to the walk in D. If a walk w in D goes through an arc more than one time in different manners, meaning that, when you go through this arc, the length of w_0 at this point is different from the previous times, this walk is transformed into a path on the auxiliary digraph. This holds because the correspondent arcs of this arc in the auxiliary digraph will be different in the two mentioned steps of the walk. It is important to notice that a walk w_1 in D that goes through the same arc, in the same manner, more than one time, is irrelevant for our goal because the walk without the loop will have the same result with respect to the length mod 3 and a lower length. By that, all the walks we are interested in are transformed into paths in the auxiliary graph. Therefore, the shortest rs-path from $(r, 0) \to (s, 1)$ on the auxiliary graph is the shortest rs-walk on D with length $l \equiv 1 \pmod 3$. By running a Breadth-First-Search on the auxiliary digraph you find this desired object.

Finally, to get the walk in D, the function Relabeling(p_{aux}) receives a path on the auxiliary digraph and relabel all the vertices and arcs to match with the original graph. If a vertex has form $(v, i), i \in \{0, 1, 2\}$, it will be relabeled to v. An edge from $((v, i), (u, j)), i \in \{0, 1, 2\}, j \in \{0, 1, 2\}$, will be relabeled to (v, u). This will ensure that the rs-path is the according rs-walk in the original graph.

Solution 4. First, it is impossible to have both objects at the same time. If you have a matching that saturates A, you have $|A|$ different edges, one from each element in A to a different element in B. Therefore, if there is a matching that saturates A, the inequality $|S| \leq |N(S)|$ holds for every $S \subseteq A$ because every $S \subseteq A$ has at least $|S|$ elements in B that are neighbors of S. Also, if you have a subset $S \subseteq A$ such that $|S| > |N(S)|$ you can not have a matching that saturates A because there is a vertex v in S such that v is not an end of any edge on a maximum matching of G. This is easy to see because you can not have a matching that saturates S if you have fewer neighbors of S than $|S|$. If you can not have a matching that saturates S, you can not have a matching that saturates A.

The following algorithm solves the problem.

Algorithm 3 Matching-or-Hall-Set $(G, (A, B))$

```
1: M, K ← Konig's-Algorithm (G, (A, B))
2: if |M| = |A| then
3:     return M
4: end if
5: S ≔ ∅                          ▷ S will be the subset S ⊆ A such that |S| > |N(S)|
6: for each v ∈ A do
7:     if v ∉ K then              ▷ this can be done in constant time by using an auxiliary array
8:         S ← S ∪ v
9:     end if
10: end for
11: return S
```

By corollary 20, the call of Konig's algorithm with a bipartite graph returns (M, K), where M is a maximum matching of G and K is a minimum vertex cover of G, and $|M| = |K|$.

The algorithm 3 runs Konig's algorithm, checks if M saturates A, and returns M if the verification is true. Otherwise, we know that no matching of G saturates A, and we need to return a subset $S \subseteq A$ such that $|S| > |N(S)|$. In algorithm 3 $S \coloneqq \{a \in A : a \notin K\}$.

First, S is a subset of A. Second, $N(S)$ is the set of all elements $b \in B$ such that $b \in K$. If $b \in B$ and $b \in K$, b has at least one neighbor that is not in K, otherwise K would not be a minimum vertex cover because you could remove b from K, and it would still be a vertex cover, what is a contradiction. Therefore, b has a neighbor in S and, by definition, is in $N(S)$. Finally, now that we know $N(S)$, we will prove that $|S| > |N(S)|$. An important property of this setup is that $|M| = |K|$ and $|M| < |A|$, thus $|K| < |A|$. Let $K_A \coloneqq \{a \in A : a \in K\}$ and $K_B \coloneqq \{b \in B : b \in K\}$. K_A and K_B are disjoint so we have $|K_A| + |K_B| < |A|$. Working on that, trying to have an inequality where one side is a subset of A, $|K_B| < |A| - |K_A|$. Because $K_A \subseteq A$, $|A| - |K_A| = |A \setminus K_A|$. Knowing that $K_B = N(S)$ and $A \setminus K_A = S$, it is proved that $|N(S)| < |S|$.

Regarding the time analysis, in line 1, we run Konig's algorithm, which was analyzed in class. Lines 2, 3, and 4 can be done in constant time if you store the size of the set (in the worst case, the IF verification is linear on the number of elements of the sets). The FOR loop on line 5 runs $|A|$ times, and the IF on line 6 can be done in constant time by using an auxiliary boolean array that stores whether a vertex v is in K or not (it costs linear time on $|V(G)|$ to build the array plus linear time on $|K|$ to fill the array). Concluding, the time of Konig's algorithm defines the time of the algorithm 3, $(O(|V(G)|(|V(G)| + |E(G)|)))$.